

Vector Representation

Suggested time: 30 seconds.

Script:

Today we move from text as raw symbols to text as vectors. The core question for the first half is: how can a machine represent the meaning of a word or document numerically?

Keep the framing simple: everything we discuss today is about turning language into numbers so that similarity, classification, and language modelling become possible.

Pacing note: Aim for about 2 hours of teaching, then use the remaining class time for lab, discussion, and assessment questions.

Word embedding - intro



Suggested time: 2 minutes.

Script:

This opening image is a useful mental model. When people talk about what is inside an LLM, one of the key ingredients is a large collection of vectors. A model does not directly understand the word “dog” or “bank” the way humans do. It stores and manipulates numerical representations. These vectors are not only for words; modern systems also build vectors for sentences, documents, images, and other modalities.

For this lecture, we start with the simplest historical path: word embeddings. Once students understand word embeddings, it becomes much easier to understand why modern LLMs rely so heavily on representation learning.

How to represent a word or a document?

Free entry in 2 a wkly
comp to win FA Cup final
tkts 21st May 2005. Text
FA to 87121 to receive
entry question(std txt
rate)T&C's apply
08452810075over18's

→ Ham or Spam

Suggested time: 3 minutes.

Script:

Before we jump into embeddings, start with the practical problem. Suppose this SMS comes into a classifier. We want the system to decide whether it is ham or spam.

The text contains obvious spam signals: “free entry,” “win,” “text FA,” phone numbers, and terms and conditions. But a machine learning model cannot directly operate on raw text. It needs a numerical representation.

So the question is: what should the input vector look like? Should we count words? Should we use a dictionary? Should we represent every word separately? Or should we represent the whole message as one vector?

This slide sets up the rest of the lecture: representation is not a side detail; it strongly determines how well the downstream classifier can work.

Word Semantics and Static Embeddings

How do we represent what a word means?

Suggested time: 30 seconds.

Script:

This is the first main block: word semantics and static embeddings. Static means that each word gets one fixed vector, regardless of the sentence it appears in.

The guiding question is: how do we represent what a word means? We will move from symbolic definitions to distributional vectors, and then to neural word embeddings such as Word2Vec and GloVe.

The Meaning Problem

- How do you represent what a word means to a machine?
- Symbolic representations -- WordNet
 - Organises words into a hierarchy of meanings (synsets): car -> vehicle -> artifact -> ...
 - Hand-constructed, expensive to maintain, no notion of degree of similarity
- One-hot encoding
 - "cat" = [1,0,0,...], "dog" = [0,1,0,...] -- cosine similarity between any two words is 0
 - No notion of similarity at all

Suggested time: 3 minutes.

Script:

The meaning problem is deceptively simple: how do we represent the meaning of a word to a machine?

One historical answer is symbolic representation, such as WordNet. WordNet organises words into synsets and hierarchical relations. For example, car is a kind of vehicle, vehicle is an artifact, and so on. This is precise and human-interpretable, but it is expensive to build and maintain.

Another basic answer is one-hot encoding. Each word gets a long vector with one position switched on. The problem is that cat and dog are just as different as cat and refrigerator. Their cosine similarity is zero because the vectors are orthogonal.

So symbolic systems are meaningful but rigid; one-hot vectors are easy but semantically empty. Embeddings try to solve this gap.

Surface form definition

Introductory logic classes:

- ❑ The meaning of "dog" is DOG; cat is CAT
- ❑ $\forall x \text{ DOG}(x) \rightarrow \text{MAMMAL}(x)$

Surface definition is rigid and rely on expert annotation.

We would like a more flexible way of sense making → **vector semantics**

Word 1	Word 2	Similarity
vanish	disappear	9.8
behave	obey	7.3
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

SimLex-999 dataset (Hill et al., 2015)

Suggested time: 3 minutes.

Script:

This slide shows why surface-form definitions are limited. In logic, we can say the meaning of dog is DOG, and if something is a DOG then it is a MAMMAL. That works nicely in a controlled symbolic setting.

But it relies on expert annotation and rigid categories. It does not naturally tell us that vanish is very close to disappear, or that hole and agreement are almost unrelated.

The SimLex-style similarity table is a good bridge. Humans can assign degrees of similarity. We would like a machine representation that can also express degrees of relatedness, not just exact symbolic membership.

This motivates vector semantics: words are represented as points in a space, and similarity becomes distance or angle in that space.

Why do we need embedding?

- Analogical relations are secondary
- We need a continuous representation
 - for machine learning
 - **for similarity comparison**
 - knowing sofa and couch refer to the same concept is crucial
 - for word sense disambiguation
 - to break away from the surface form

Suggested time: 2 minutes.

Script:

The main point here is that analogies are interesting, but they are not the primary reason we need embeddings.

The practical reason is continuous representation. Machine learning models work naturally with numbers, and continuous vectors allow similar words to have similar representations.

For example, sofa and couch may have different surface forms, but they refer to almost the same concept. A good representation should help a classifier or search system treat them as related.

This is also important for word sense disambiguation and for breaking away from simple string matching.

The Distributional Hypothesis

- "You shall know a word by the company it keeps." -- Firth (1957); Harris (1954)
 - Words that appear in similar contexts have similar meanings
- From hypothesis to vectors
 - Build a word-context co-occurrence matrix
 - Rows = words, columns = context words, values = how often word w appears near context word c

Suggested time: 3 minutes.

Script:

The distributional hypothesis is the foundation of classical vector semantics: you shall know a word by the company it keeps.

In simple terms, words that occur in similar contexts tend to have similar meanings. If two words frequently appear near the same surrounding words, we expect them to be semantically related.

To turn this into vectors, we build a word-context matrix. Rows are target words, columns are context words, and each cell records how often a target word appears near a context word.

This is the key bridge from text to numbers: meaning is approximated by linguistic distribution.

Defining meaning by linguistic distribution

□ Tf-idf, counting-based vectorizer

	call	text	free	win	...	...	let's	go
email_0								
email_1								
email_2								
email_3								
email_4								
...								
...								
email_n								

Suggested time: 2.5 minutes.

Script:

Here we connect the idea to a familiar representation: TF-IDF or count-based vectorization.

For document classification, each row might be an email and each column might be a word such as call, text, free, win, let's, or go. The values can be raw counts, binary presence, or TF-IDF weights.

This is already a vector representation. A spam email may have high values for words like free, win, text, or call. A normal message may have different words.

The limitation is that this representation is tied to surface words. It does not automatically know that prize and reward may be related unless both were observed in similar statistical patterns.

Co-occurrence Matrix -- A Toy Example

	eats	meows	barks	purrs	runs
cat	5	12	1	10	8
dog	6	0	15	0	9
car	0	0	0	0	7



"cat" and "dog" have similar row vectors; "car" is very different -- semantic similarity emerging from statistics.

Suggested time: 3 minutes.

Script:

Use this toy table to make the intuition concrete. The row for cat has high counts for meows and purrs. The row for dog has high counts for barks and runs. The row for car is very different.

Even in this tiny example, cat and dog are more similar to each other than either is to car because they share some animal-like contexts such as eats and runs.

The important idea is that we did not explicitly tell the machine that cat and dog are animals. Similarity emerges from the statistics of the contexts.

This is the central idea behind distributional semantics.

Word-Context Matrix

- each row represents a word in the vocabulary
- each column represents how often each other word in the vocabulary appears nearby

	aardvark	abacus	adept	affect	agate	...	zydeco
aardvark							
abacus							
adept							
affect							
agate							
...							
zydeco							

How often does **agate** occur near **abacus**?

Suggested time: 2 minutes.

Script:

This slide shows the general form of the word-context matrix. Each row is a word in the vocabulary. Each column is another word that may appear nearby.

The highlighted question is: how often does agate occur near abacus? Most such pairs will have zero or very small counts, which is why these matrices become extremely sparse.

Still, each row can be treated as a vector. The meaning of a word is represented by the pattern of context counts across all columns.

The word-context matrix

Let's say within 4 words is considered nearby

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

Suggested time: 3 minutes.

Script:

Here we define what “nearby” means. In this example, words within a window of four words are treated as context.

Look at cherry. In the phrase “cherry pie,” pie occurs nearby. So the count for cherry with the context word pie increases. Strawberry also appears near pie in the second example.

Digital appears near computer, and information appears near computer and internet-related words.

The specific window size is a modelling choice. A small window captures local syntactic or functional similarity; a larger window captures broader topical similarity.

The word-context matrix

Let's say within 4 words is considered nearby

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Suggested time: 3 minutes.

Script:

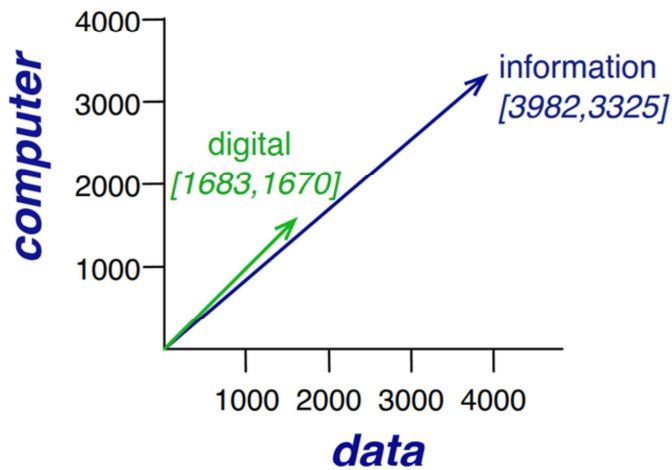
This slide extends the previous idea from a few toy contexts to a larger corpus.

Now the counts are much larger. Digital has high counts with computer and data. Information also has high counts with computer and data. Cherry has a high count with pie.

So we expect digital and information to be closer in this representation than cherry and information. Not because we gave the model a dictionary, but because their context profiles are similar.

This is the statistical basis for measuring word similarity.

Linguistic distribution determines word similarity



$$\text{cosine}(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

Suggested time: 3 minutes.

Script:

Now we move from the table to geometry. Each word row can be seen as a vector. In this simplified plot, we only show two dimensions: data and computer.

Digital and information point in a similar direction because both have large values for data and computer. Cosine similarity measures this direction similarity.

The key point is that cosine similarity is about the angle between vectors, not just their raw length. This is useful because common words may have larger counts overall, but direction tells us whether the pattern of contexts is similar.

Linguistic distribution determines word similarity

$$\cos(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry}, \text{information}) =$$

$$\frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .017$$

$$\cos(\text{digital}, \text{information}) =$$

$$\frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

Suggested time: 3 minutes.

Script:

This slide works through the actual cosine computation.

For cherry and information, the numerator is the dot product of their vectors.

But the denominator divides by the length of each vector. After this normalization, the similarity is very low, around 0.017.

For digital and information, the context counts align strongly in the data and computer dimensions, so the cosine similarity is very high, around 0.996.

The practical takeaway: if two words have similar context distributions, their vectors point in similar directions, and cosine similarity becomes high.

From Raw Counts to PMI (optional)

- Problem: raw co-occurrence counts are dominated by frequent words ("the", "a", "in")
 - A high count for "cat" co-occurring with "the" says little about cat's meaning
- Pointwise Mutual Information (PMI) upweights surprising co-occurrences
 - $PMI(w, c) = \log(P(w, c) / [P(w) \times P(c)])$
 - High PMI: "cat" and "meows" co-occur more than chance predicts -> strong association
 - Low PMI: "cat" and "the" co-occur, but "the" co-occurs with everything -> weak association

Suggested time: 2 minutes. Optional if time is tight.

Script:

Raw counts have a problem: frequent words dominate. Words like “the,” “a,” and “in” appear near almost everything. A high co-occurrence count with “the” does not tell us much about meaning.

PMI, or pointwise mutual information, asks whether a co-occurrence is surprising compared to chance. Cat and meows may co-occur more than expected, so that association is informative. Cat and the may co-occur often, but the occurs with everything, so the association is weak.

You can treat this as an optional deeper statistical refinement before moving to dense vectors.

Sparse versus dense vectors

Count vectors (even if weighted by tf-idf)

- long (length $|V| = 20,000$ to $50,000$)
- sparse (most elements are zero)

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Alternative: learn/compute vectors which are

- short (length 50-1000)
- dense (most elements are non-zero)

Suggested time: 2 minutes.

Script:

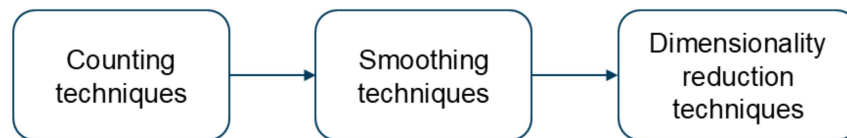
Count vectors and TF-IDF vectors are often long and sparse. If the vocabulary has 50,000 words, each vector has 50,000 dimensions, and most entries are zero.

Dense vectors are the alternative. Instead of tens of thousands of dimensions, we use maybe 50, 100, 300, or 1000 dimensions. Most entries are non-zero.

Dense embeddings are easier for neural models to use, cheaper to store, and better at capturing graded similarity. The next question is how to obtain these dense vectors.

Statistical method

- ❑ Statistical method (based on calculation) can derive dense vectors
- ❑ Statistical method provides a foundation for understanding “how meaning is represented in vectors”
- PPMI (performed on the word-context matrix)
- PCA (dimensionality reduction technique)
- SVD (dimensionality reduction technique)



Suggested time: 2 minutes.

Script:

There are two broad routes to dense vectors. The first is statistical: start with a word-context matrix, apply weighting such as PPMI, and then reduce dimensionality using PCA or SVD.

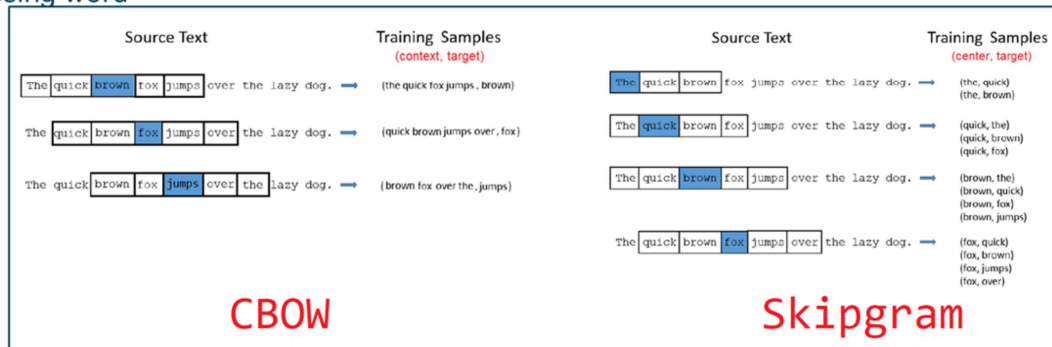
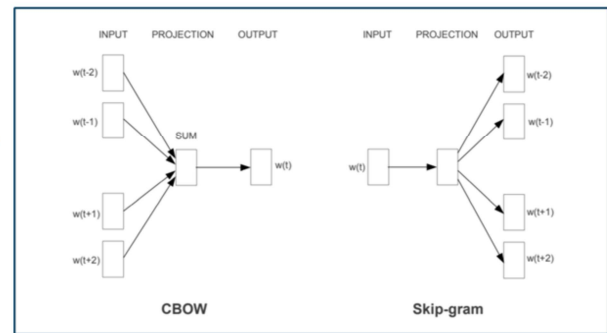
The pipeline is counting, then smoothing or weighting, then dimensionality reduction.

The details of PCA and SVD are not the focus here, but the intuition is important: we compress a very large sparse matrix into a smaller dense space while preserving important similarity structure.

This route helps students understand that embeddings are not magic; they are compressed statistical patterns.

Neural Network based

- ❑ NN-based vector representations are learnt through a shallow language model*
- ❑ The objective is a bit different,
 - not predicting the next word
 - but predicting the surrounding word
 - or use surrounding words to predict the missing word



Suggested time: 3 minutes.

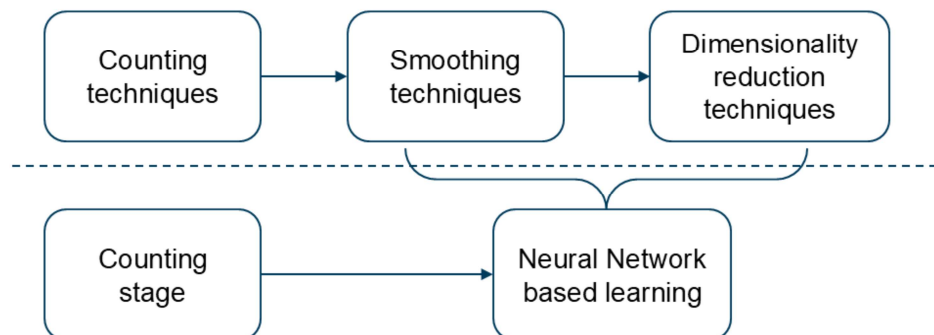
Script:

The second route is neural-network-based embeddings. Word2Vec is the classic example.

Instead of explicitly building and reducing a huge matrix, we train a shallow neural network on a prediction task. The vectors emerge as learned weights. There are two common objectives. In CBOW, the model uses surrounding context words to predict the centre word. In skip-gram, the model uses the centre word to predict surrounding context words.

The goal is not primarily to build a next-word generator. The goal is to learn useful word vectors from these prediction tasks.

Difference Between Statistical vs Neural Network based



Not explicitly taking down the number of occurrences of neighbour words, but implicitly performed through data preparation and sampling.

Suggested time: 2.5 minutes.

Script:

This slide compares the statistical and neural views.

In the statistical approach, we explicitly count co-occurrences, apply smoothing or weighting, and then reduce dimensionality.

In the neural approach, the counting is implicit. When we prepare training samples from windows of text, frequent word-context pairs appear more often. The network learns from these samples and adjusts the embedding weights.

So the two approaches are not completely unrelated. Both are driven by word-context statistics. The difference is whether we explicitly build the matrix or let the neural objective learn from sampled contexts.

Word2Vec: Learning Embeddings from a Shallow Network

- The co-occurrence matrix is huge (vocab x vocab) and sparse -- we need dense, low-dimensional vectors
- Word2Vec (Mikolov et al. 2013): train a shallow network on a prediction task; embeddings emerge as its weights
 - CBOW (Continuous Bag of Words)
 - Given the context words, predict the centre word
 - Skip-gram
 - Given the centre word, predict each context word

Suggested time: 3 minutes.

Script:

Word2Vec addresses the problem that the full co-occurrence matrix is huge and sparse.

The idea from Mikolov and colleagues is to train a shallow network on a simple prediction task. The useful product is not only the prediction accuracy; it is the embedding matrix learned during training.

CBOW predicts the centre word from the surrounding words. Skip-gram predicts surrounding words from the centre word.

A simple mnemonic: CBOW collects the bag of surrounding words and predicts the centre. Skip-gram starts with the centre word and skips out to predict the context.

Negative Sampling

- A full softmax over the vocabulary is expensive -- normalising over 50,000 words every step
- Negative sampling approximates it:
 - For each real (word, context) pair, sample k random words that are NOT the context
 - Train the model to predict "yes" for the real pair, "no" for the random ones
 - Typically k = 5-20 -- this makes training tractable

Suggested time: 3 minutes.

Script:

A practical issue in Word2Vec is the output layer. If the vocabulary has 50,000 words, a full softmax means scoring and normalising across all 50,000 possible words at every training step. That is expensive.

Negative sampling makes the task simpler. For a real pair like dog-barks, we label it positive. Then we sample a few random words that are not the true context, such as dog-table or dog-democracy, and label them negative.

The model learns to distinguish real word-context pairs from random ones.

This makes training much faster and is one reason Word2Vec became practical at scale.

Spatial Learning

<https://drive.google.com/file/d/1t9pT5allnOQNCU4Cj4uhVeQl3mQUA1lO/view?usp=sharing>



Suggested time: 2 minutes. Optional visual break.

Script:

This slide is mainly for spatial intuition. If the linked video is available, play only a short segment. The objective is not the video itself, but the idea that embeddings create a space where related items are close and unrelated items are far apart.

If time is tight, summarize verbally: training adjusts vectors so that words appearing in similar contexts move closer together, while unrelated words tend to move apart.

Then transition to what embeddings can capture.

What Embeddings Capture

- Semantic similarity
 - king is close to queen, Paris is close to London
- Analogies -- arithmetic in embedding space gives meaningful results
 - king - man + woman $\sim$ queen
- Syntactic regularities
 - walked - walk + run $\sim$ ran

Suggested time: 2 minutes.

Script:

Embeddings capture several kinds of regularities.

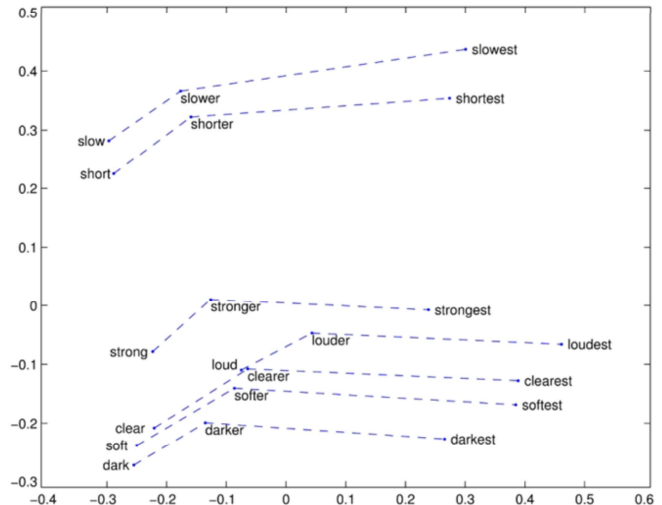
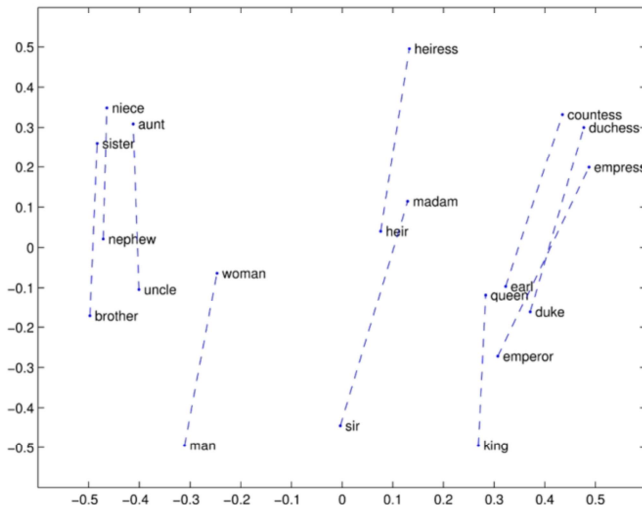
First, semantic similarity: king and queen are related; Paris and London are both capital-city-like entities.

Second, analogies: vector arithmetic can sometimes move between relations, such as king minus man plus woman being close to queen.

Third, syntactic regularities: tense and morphology can appear as consistent directions, such as walk to walked or run to ran.

Important qualification: these patterns are useful but not perfect. They are statistical regularities, not guaranteed logical rules.

Analogue relations



Suggested time: 2 minutes.

Script:

This GloVe visualization shows analogue relations as roughly parallel directions in vector space.

On the left, many gender-related pairs have similar offsets. On the right, adjective comparative and superlative relations show similar geometric patterns.

The point is not that every analogy always works. The point is that embeddings can encode relational structure in the geometry of the space. This is one reason embeddings were such a breakthrough before large contextual models became dominant.

"Paris is to France as Berlin is to ____"

Paris = [0.8, 0.1]

France = [0.3, 0.9]

Berlin = [0.7, 0.2]

Berlin - Paris + France

= [0.7-0.8+0.3, 0.2-0.1+0.9]

= [0.2, 1.0]

Nearest word to [0.2, 1.0] should be "Germany"

(toy numbers -- not real GloVe vectors)



Worked Analogy Example (Toy 2D Vectors)

Suggested time: 3 minutes.

Script:

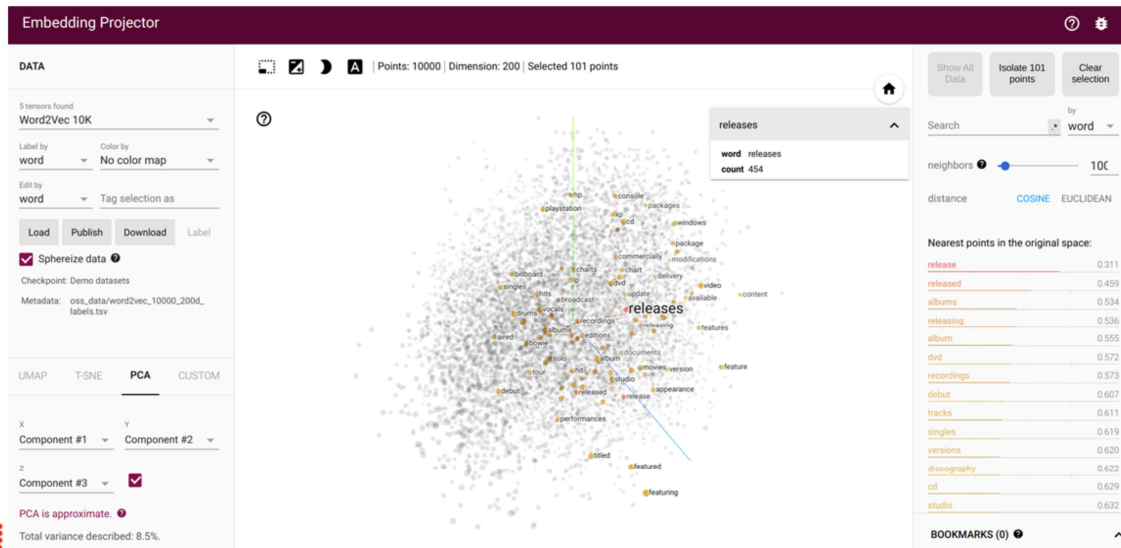
This is a toy analogy calculation. We want: Paris is to France as Berlin is to what?

The relation from Paris to France is approximately a capital-to-country relation. To apply that relation to Berlin, we compute Berlin minus Paris plus France.

With the toy numbers, the resulting vector is [0.2, 1.0], and the nearest word should be Germany.

Stress that these are not real GloVe values. The purpose is only to show the arithmetic idea: subtract one concept, add another relation, then search for the nearest vector.

Play with embedding



<https://projector.tensorflow.org/>

27

Suggested time: 4 minutes. Optional live demo.

Script:

This is a good point for a short interactive demonstration if time permits. Open TensorFlow Embedding Projector and search for a few familiar words. Suggested examples: king, queen, doctor, hospital, music, computer, Singapore. Ask students what nearest neighbours they expect before showing them.

Keep the demo short. The teaching point is that high-dimensional embeddings can be projected into 2D or 3D for exploration, but the projection is only an approximation.

If the internet or projector is unreliable, skip the live demo and simply explain the screenshot.

GloVe

- Pennington et al. (2014)
 - Combines global co-occurrence statistics (matrix approach) with local context (Word2Vec-style)
 - Objective: the dot product of two word vectors should equal the log of their co-occurrence probability
- Pre-trained GloVe vectors (6B, 42B, 840B token versions) are widely used
 - This is what you load in the TMA (glove.6B.100d.txt)

Suggested time: 3 minutes.

Script:

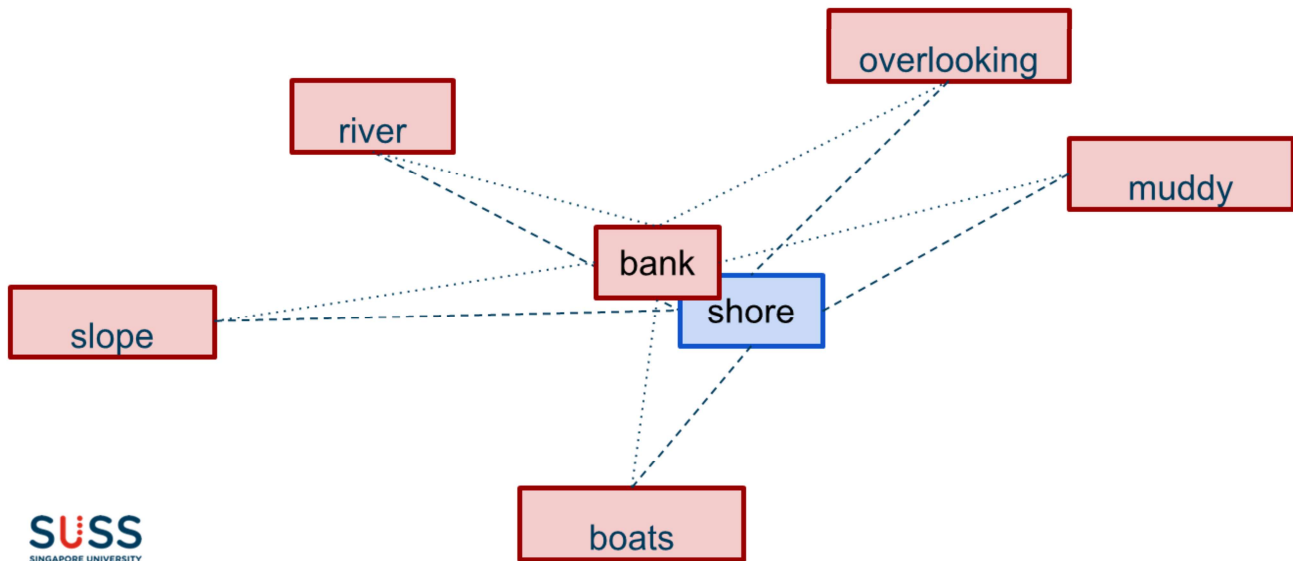
GloVe stands for Global Vectors. It was proposed by Pennington, Socher, and Manning in 2014.

GloVe combines the matrix view and the predictive embedding view. It uses global co-occurrence statistics, but learns dense vectors through an optimization objective.

A simplified way to say it is: the dot product between two word vectors should reflect the log of their co-occurrence statistics. In the actual model there are bias terms and a weighting function, so avoid presenting it as an exact plain equality.

For the TMA, students will load pre-trained GloVe vectors, such as glove.6B.100d.txt, and use them as features.

The context words (Neighbours)



Suggested time: 2 minutes.

Script:

This slide prepares the limitation of static embeddings using the word “bank.” The neighbours shown here, such as river, shore, muddy, slope, and boats, clearly suggest the river-bank sense. But the same word “bank” can also mean a financial institution.

A static embedding gives one vector to the spelling bank. It cannot create a separate vector for river bank and bank account unless we use a different modelling approach.

This is the bridge to polysemy.

What Embeddings Do Not Capture: Polysemy

- "bank" in "river bank" and "bank account" gets ONE embedding vector in Word2Vec/GloVe
 - Both meanings are averaged into a single point in space
 - Which meaning dominates depends on which usage is more frequent in the training corpus
- This directly affects classification
 - If "bank" appears in a financial article (Business) and a river-bank story (World), the embedding carries both signals and cannot distinguish -- TMA Q2c error analysis will surface this

Suggested time: 3 minutes.

Script:

Polysemy means one word form has multiple meanings. In Word2Vec and GloVe, "bank" has one embedding vector, even though it can mean river bank or financial bank.

That single vector averages evidence from all usages in the training corpus. If financial uses are more common, the vector may lean financial. If river contexts are common, it may contain more geographical signal.

This matters for classification. A word like bank could appear in a business article or in a world/news story about flooding. A static embedding alone may not disambiguate which sense is intended.

This is one major reason modern contextual embeddings are more powerful: they create context-dependent representations.

Representing a Document: Averaging Word Embeddings

- A classifier needs one vector per document, not one per word
 - Simplest approach: average the word embeddings
- $\text{doc_vector} = (\text{embed}(w_1) + \text{embed}(w_2) + \dots + \text{embed}(w_n)) / n$
- Limitations
 - Word order is lost -- "dog bites man" and "man bites dog" get the same vector
 - All words contribute equally, regardless of importance
 - OOV words contribute nothing (zero vector) -- biases documents with many rare words toward the origin

Suggested time: 3 minutes.

Script:

A classifier usually needs one vector per document. But GloVe gives one vector per word. The simplest solution is to average the word embeddings in the document.

The document vector is the mean of the word vectors: add them and divide by the number of tokens.

This is easy and often works surprisingly well as a baseline, but it has limitations. Word order is lost, so “dog bites man” and “man bites dog” look the same. All words contribute equally unless we add weighting. Out-of-vocabulary words also cause problems.

For the TMA, be careful to follow the specified handling of OOV tokens.

Cosine Similarity -- The Maths

- $\text{cosine_similarity}(u, v) = (u \cdot v) / (|u| \times |v|)$
 - $|u| = \sqrt{u_1^2 + u_2^2 + \dots + u_n^2}$ -- the L2 norm, or "length", of the vector
- Range: -1 (opposite) to 0 (unrelated) to 1 (same direction / very similar)
- Used in: today's lab, TMA Q2b, and ECA B1 if you choose the GloVe baseline

Suggested time: 3 minutes.

Script:

Cosine similarity is a recurring calculation in this lecture, the lab, and the assessment.

The formula is dot product divided by the product of the two vector norms.

Emphasize that the dot product alone is not cosine similarity.

The result ranges from -1 to 1. In many embedding applications, values closer to 1 mean very similar direction. Values near 0 mean weak or no similarity. Negative values mean opposite directions in the vector space.

The main practical message: always normalize by both vector lengths.

How to evaluate embeddings (lab)

- ❑ Intrinsic evaluation
 - analogy task
 - similarity task
 - bilingual lexicon induction (BLI)
- ❑ Extrinsic evaluation
 - downstream application

Suggested time: 2 minutes.

Script:

There are two broad ways to evaluate embeddings.

Intrinsic evaluation tests the embeddings directly. Examples include analogy tasks, similarity tasks, or bilingual lexicon induction.

Extrinsic evaluation tests whether the embeddings improve a downstream application such as classification, search, question answering, or sentiment analysis.

For students, the most important mindset is: a beautiful embedding space is useful only if it helps the actual task or gives interpretable evidence.

Common Pitfalls & Assessment Links -- Word Embeddings

- Common pitfalls
 - Computing dot product and calling it cosine similarity -- always divide by both norms
 - Dropping OOV tokens instead of using a zero vector -- changes the average's denominator
 - Mixing up CBOW and skip-gram -- skip-gram SKIPs to the context; CBOW predicts the centre
- Where this shows up in your assessments
 - TMA Q2b: average GloVe-100 embeddings per document, train logistic regression, compare to TF-IDF (Q2a)
 - ECA B1: averaged-GloVe + logistic regression is one of the four baseline options



Suggested time: 3 minutes.

Script:

This slide is very assessment-relevant. First pitfall: computing only the dot product and calling it cosine similarity. That is wrong unless the vectors are already normalized.

Second: OOV handling. In the TMA, if the instruction says use a zero vector for OOV words, do that consistently. Dropping OOV words changes the denominator of the average and changes document vectors.

Third: do not mix up CBOW and skip-gram. CBOW predicts the centre from context; skip-gram predicts context from the centre.

Connect this directly to TMA Q2b and the ECA baseline option involving averaged GloVe plus logistic regression.

Lab activity 1

1. Train an embedding
2. Use that embedding to initialize a language model

Suggested time: 5 minutes for setup; actual lab can continue after lecture.

Script:

This is the first lab activity. The two tasks are: train an embedding, and then use that embedding to initialize a language model.

Before students start, clarify the objective. We are not trying to train a production-quality embedding. We are trying to see how the concepts from the lecture become code.

Ask students to watch for three things: how training samples are created from context windows, where the embedding matrix appears in the model, and how the learned vectors are reused in the language model.

If time is limited, demonstrate the expected workflow and let students complete the rest during the lab portion.



Statistical Language Models

N-grams, smoothing, and perplexity

Suggested time: 30 seconds.

Script:

Now we switch to the second main block: statistical language models.

The bridge is natural. Embeddings represent words and documents.

Language models assign probabilities to word sequences.

We will start with n-grams, smoothing, and perplexity because these ideas remain useful even when studying modern neural language models.

What Is a Language Model?

- A probability distribution over sequences of words: $P(w_1, w_2, \dots, w_n)$
- What this enables
 - Text generation -- sample from the distribution
 - Scoring -- "is this fluent English?"
 - Classification (TMA Q3) -- assign a document to the class whose LM scores it highest
- The chain rule of probability
 - $P(w_1, \dots, w_n) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \times \dots \times P(w_n|w_1, \dots, w_{n-1})$

Suggested time: 3 minutes.

Script:

A language model is a probability distribution over sequences of words. It tells us how likely a sequence is: $P(w_1, w_2, \dots, w_n)$.

This supports three important functions. First, generation: sample likely next words. Second, scoring: judge whether a sentence is fluent or likely. Third, classification: choose the class-specific language model that assigns the highest probability to a document.

The chain rule decomposes a full sentence probability into a product of conditional probabilities. The challenge is that the full history becomes too large to estimate directly.

The Markov Assumption

- Exact chain-rule computation needs to have seen the full history -- almost never happens
- Approximate: the next word depends only on the last $k-1$ words
 - $k=1$ (unigram): independent of context
 - $k=2$ (bigram): depends only on the immediately preceding word
 - $k=3$ (trigram): depends on the two preceding words
- Maximum likelihood estimation
 - $P(w_n | w_{n-1}) = C(w_{n-1}, w_n) / C(w_{n-1})$

Suggested time: 3 minutes.

Script:

The Markov assumption solves the full-history problem by using only a limited recent context.

A unigram model ignores context. A bigram model uses one previous word.

A trigram model uses two previous words.

For a bigram model, the probability of the next word is estimated by counts: count of the bigram divided by count of the previous word.

This is maximum likelihood estimation. It is simple and interpretable, but it depends heavily on what has been observed in the training data.

Training sentence: "the cat sat on the mat"

$P(\text{sat} \mid \text{cat}) = \text{count}(\text{"cat sat"}) / \text{count}(\text{"cat"})$

If "cat" appears 10 times and "cat sat" appears 2 times:

$P(\text{sat} \mid \text{cat}) = 2 / 10 = 0.2$

The remaining 0.8 is distributed over other words
that can follow "cat" -- "meows", "runs", "slept", ...



Worked Example -- Bigram Probability

Suggested time: 3 minutes.

Script:

Work through this example slowly. The training sentence is "the cat sat on the mat," but imagine a larger corpus where cat appears 10 times and cat sat appears 2 times.

The bigram probability $P(\text{sat} \mid \text{cat})$ is $\text{count}(\text{cat sat})$ divided by $\text{count}(\text{cat})$, so 2 divided by 10 equals 0.2.

The remaining probability mass is distributed over other words that can follow cat: meows, runs, slept, and so on.

This example makes clear that n-gram probabilities are empirical estimates from observed counts.

The Sparsity Problem

- If a bigram never occurred in training, MLE assigns it probability 0
 - A single zero-probability bigram makes the entire sentence probability 0
 - The model cannot score any sentence containing an unseen bigram -- breaks classification
- But language is creative
 - "the dog ate the steak" is grammatical even if "dog ate" never appeared in training

Suggested time: 3 minutes.

Script:

The sparsity problem is the main weakness of maximum likelihood n-gram models.

If a bigram never occurred in training, MLE assigns it probability zero. Then any sentence containing that bigram gets probability zero, even if the sentence is perfectly grammatical.

For example, "the dog ate the steak" may be reasonable, but if dog ate never appeared in the training data, the model cannot score it properly.

This is disastrous for classification because one unseen bigram can wipe out the probability of the whole document. That is why smoothing is necessary.

Add-1 (Laplace) Smoothing

- $P(w_n | w_{n-1}) = (C(w_{n-1}, w_n) + 1) / (C(w_{n-1}) + V)$
 - V = vocabulary size (total unique tokens in the training corpus)
 - +1 in the numerator: pretend every bigram was seen at least once
 - + V in the denominator: normalise correctly across all V possible next words
- Critical for the TMA
 - V must be the GLOBAL vocabulary size, not the per-class vocabulary -- otherwise per-class LMs (TMA Q3a) are not comparable

Suggested time: 4 minutes.

Script:

Add-1 or Laplace smoothing is the simplest fix. We pretend every possible bigram has been seen once.

The numerator becomes count of the bigram plus 1. The denominator becomes count of the previous word plus V , where V is the vocabulary size. The plus V is important because there are V possible next words, and we added one pseudo-count to each of them.

For the TMA, emphasize the critical detail: V must be the global vocabulary size, not a separate per-class vocabulary size. Otherwise different class language models are not comparable.

```
Vocabulary V = 5: {the, cat, sat, on, mat}
"the" appears 4 times; "the cat"=3, "the mat"=1

P(cat | the) = (3+1)/(4+5) = 4/9 ~ 0.44
P(mat | the) = (1+1)/(4+5) = 2/9 ~ 0.22
P(sat | the) = (0+1)/(4+5) = 1/9 ~ 0.11  <- was zero before smoothing

Check: sum of P(w | the) over all 5 vocab words = 1.0
```



Suggested time: 4 minutes.

Script:

This worked example shows how smoothing changes probabilities.

The vocabulary has five words. The word “the” appears four times. The bigram “the cat” appears three times, and “the mat” appears once.

Without smoothing, $P(\text{sat} | \text{the})$ would be zero because “the sat” was not observed. With add-1 smoothing, it becomes 1 divided by 9.

Check the denominator: $\text{count}(\text{the})$ is 4, vocabulary size is 5, so the denominator is 9. The probabilities across all five possible next words sum to 1.

This is the practical sanity check students should use in their implementation.

Other Smoothing Approaches

- Add-k smoothing
 - Add k instead of 1; k can be tuned on a development set
- Backoff
 - If a trigram is unseen, fall back to bigram; if that's unseen, fall back to unigram
- Interpolation
 - Combine unigram, bigram, trigram probabilities with learned weights
- TMA Q3 requires add-1 smoothing specifically

Suggested time: 2 minutes.

Script:

There are many smoothing methods beyond add-1.

Add-k uses a smaller value than 1, which can be tuned. Backoff uses a lower-order model when a higher-order n-gram is unseen. Interpolation combines unigram, bigram, and trigram probabilities with weights.

For this course and TMA Q3, keep the implementation focus on add-1 smoothing. Mention the others only to show that smoothing is a broader family of techniques.

Perplexity

- $PP(W) = \exp(-(1/N) * \text{sum of } \log P(w_i | w_{i-1}))$
 - Intuition: perplexity = 100 means the model is as confused as choosing uniformly among 100 words
 - Lower perplexity = better model
- TMA Q3c -- the 4x4 perplexity matrix
 - Each per-class LM evaluated on held-out documents from each class
 - Diagonal should be lowest (a class's own LM explains its own documents best)
 - High off-diagonal overlap predicts high classification confusion between those classes

Suggested time: 4 minutes.

Script:

Perplexity measures how surprised or confused a language model is on a test sequence.

The formula uses the average negative log probability and exponentiates it. Lower perplexity means the model assigns higher probability to the observed text, so it is better at explaining the data.

A useful intuition: perplexity of 100 means the model behaves roughly as if it is choosing uniformly among 100 plausible next words.

For TMA Q3c, students will build a 4 by 4 perplexity matrix. Each class-specific language model is evaluated on documents from each class. Ideally, the diagonal values should be lowest.

The Ceiling of N-gram Models

- No generalisation across similar words -- "the cat sat" and "the dog sat" are totally independent
- Fixed context window -- bigrams see 1 previous word; long-range dependencies are invisible
- Exponential data requirements -- V^n possible n-grams; cannot observe a meaningful fraction
- This motivates the neural language model
 - Shared embeddings let the model generalise across similar words -- exactly the gap the upcoming linear/softmax and RNN sections start to close

Suggested time: 3 minutes.

Script:

This slide explains the ceiling of n-gram models.

First, they do not generalize across similar words. "The cat sat" and "the dog sat" are treated as independent events unless both are observed.

Second, the context window is fixed. A bigram model sees only one previous word, so long-range dependencies are invisible.

Third, the number of possible n-grams grows exponentially with n, so we cannot observe enough data.

This motivates neural language models, where shared embeddings let the model generalize across similar words and learn richer context patterns.

Common Pitfalls

- Common pitfalls
 - Using per-class vocabulary size as V instead of the global vocabulary -- makes LMs incomparable
 - Mixing log bases ($\log_2$ vs $\ln$) -- pick one and be consistent through the perplexity formula
 - Normalising perplexity by document count instead of bigram count
- Where this shows up in your assessments
 - TMA Q3a/b/c: per-class bigram LMs, log-likelihood classification, and the 4x4 perplexity matrix

Suggested time: 3 minutes.

Script:

This is another assessment-focused slide.

First, use the global vocabulary size for V . Per-class V makes the language models incomparable.

Second, keep the log base consistent. Natural log and log base 2 can both be used in principle, but do not mix them inside one calculation.

Third, normalize perplexity by the number of predicted tokens or bigrams, not by the number of documents.

Connect this directly to TMA Q3a, Q3b, and Q3c: per-class bigram LMs, log-likelihood classification, and the 4 by 4 perplexity matrix.

Reference

<https://web.stanford.edu/~jurafsky/slp3/>

Suggested time: 30 seconds.

Script:

The main reference for today is Jurafsky and Martin's Speech and Language Processing, especially the chapters on vector semantics, embeddings, n-gram language models, smoothing, and evaluation.

Close by reminding students of the big picture: today's classical ideas are not obsolete. Embeddings, probability, smoothing, and evaluation are still the conceptual foundation under modern neural NLP and LLMs.